

Give me the prompt examples of COT(Chain of Thought) and TOT - Logistics Domain Software Developer with Java

Absolutely. For a **Logistics Domain Software Developer using Java**, you can use **CoT-style prompting** for structured reasoning and **ToT (Tree of Thoughts)** for exploring multiple solution approaches.

Note: For production AI prompting, it is better to ask for a concise reasoning summary, assumptions, decisions, and validation rather than requesting the model to expose its private chain-of-thought.

1. CoT — Chain-of-Thought Prompt Example

Scenario: Shipment Tracking API

Prompt:

 Edit



Act as a Senior Java Spring Boot Developer specializing in Logistics and Transportation Management Systems.

Business requirement:

Build a Shipment Tracking REST API that allows a customer to track a shipment using its tracking number.

Requirements:

1. Tracking number must be unique.
2. Shipment statuses are:
 - CREATED
 - PICKED_UP
 - IN_TRANSIT
 - OUT_FOR_DELIVERY

- DELIVERED
 - CANCELLED
3. Invalid status transitions must be rejected.
 4. Every status change must be recorded in shipment history.
 5. API must return appropriate HTTP status codes.
 6. Use Java 21 and Spring Boot.
 7. Use PostgreSQL and Spring Data JPA.
 8. Include validation, exception handling, logging, and unit tests.

Analyze the requirement systematically.

Provide your response in these sections:

1. Business rules
2. Assumptions
3. Domain model
4. Shipment status transition rules
5. Database design
6. REST API design
7. Service-layer design
8. Java/Spring Boot implementation
9. Exception handling
10. Unit test strategy
11. Edge cases
12. Production considerations

For each design decision, briefly explain the rationale.

Do not expose private chain-of-thought. Provide only concise reasoning summaries and conclusions.

Expected reasoning pattern

The model should arrive at something like:

Requirement → Business rules → Domain model → State transition → Database → API → Service → Tests → Edge cases

For example:

CREATED



```
PICKED_UP
  ↓
IN_TRANSIT
  ↓
OUT_FOR_DELIVERY
  ↓
DELIVERED
```

And reject transitions such as:

```
DELIVERED → IN_TRANSIT
CANCELLED → PICKED_UP
CREATED → DELIVERED
```

2. CoT Example — Logistics Route Optimization

Scenario

You need to assign deliveries to trucks.

Prompt:

 Edit



Act as a Logistics Domain Architect and Senior Java Developer.

Problem:

A logistics company has:

- 100 shipments
- 10 delivery vehicles
- Multiple delivery locations
- Vehicle capacity constraints
- Delivery time windows
- Driver working-hour constraints

Design a Java-based solution for assigning shipments to vehicles.

Analyze the problem systematically:

1. Identify business constraints.
2. Define the domain objects.
3. Define the optimization objective.

4. Identify the required algorithms or heuristics.
5. Compare possible implementation approaches.
6. Select the most appropriate approach.
7. Design the Java classes/interfaces.
8. Design the REST APIs.
9. Explain persistence requirements.
10. Explain how the solution should scale to 100,000 shipments.
11. Identify failure and edge cases.
12. Provide a testing strategy.

Do not provide private chain-of-thought. Give concise reasoning summaries for each decision.

3. ToT — Tree of Thoughts Prompt

ToT is useful when there are multiple possible solutions.

For example, suppose you need to design a **real-time shipment tracking system**.

Instead of immediately choosing Kafka, Redis, or WebSocket, ask the AI to explore several alternatives.

Prompt

 Edit



Act as a Principal Java Architect specializing in Logistics Software.

Design a real-time Shipment Tracking Platform using Java and Spring Boot.

Business requirements:

- Track millions of shipments.
- Receive GPS/location events from vehicles.
- Process shipment status changes.
- Notify customers when shipment status changes.
- Maintain shipment history.

- Support real-time tracking.
- Handle duplicate and out-of-order events.
- Provide high availability.
- Support horizontal scaling.

Explore three architecture alternatives:

Branch A:

Spring Boot + REST + PostgreSQL + Redis

Branch B:

Spring Boot + Kafka + PostgreSQL + Redis

Branch C:

Spring Boot + Kafka + PostgreSQL + Redis + WebSocket

For each branch evaluate:

1. Architecture
2. Data flow
3. Scalability
4. Reliability
5. Event ordering
6. Duplicate event handling
7. Performance
8. Operational complexity
9. Development complexity
10. Cost considerations
11. Failure scenarios
12. Suitability for millions of shipments

Score each approach from 1–10.

Then:

1. Eliminate weak approaches.
2. Compare the remaining approaches.
3. Select the best architecture.
4. Explain the decision using concise reasoning summaries.
5. Produce the final architecture.
6. Provide Java/Spring Boot component design.
7. Provide Kafka topic design if applicable.
8. Provide database schema.

9. Provide API contracts.
10. Provide testing strategy.

Do not expose private chain-of-thought. Show only concise evaluation criteria, scores, trade-offs, and the final decision.

4. ToT Example — Order Allocation

This is particularly useful in logistics.

Business problem

A customer places an order containing 20 products.

The company has warehouses:

Warehouse A → Chennai
Warehouse B → Bangalore
Warehouse C → Hyderabad
Warehouse D → Mumbai

The system must decide **which warehouse should fulfill the order**.

ToT Prompt

 Edit



Act as a Senior Logistics Solution Architect and Java Developer.

Design an Order Allocation Engine for a logistics platform.

Input:

Customer:

- Location: Chennai

Order:

- Product A × 2
- Product B × 1

- Product C × 3

Warehouses:

- Chennai
- Bangalore
- Hyderabad
- Mumbai

Each warehouse has:

- Inventory availability
- Processing capacity
- Shipping cost
- Distance
- Estimated delivery time

Explore these allocation strategies:

Strategy A:

Nearest warehouse

Strategy B:

Lowest shipping cost

Strategy C:

Fastest delivery

Strategy D:

Hybrid scoring algorithm

For each strategy:

1. Explain the algorithm.
2. Identify advantages.
3. Identify disadvantages.
4. Identify failure scenarios.
5. Estimate computational complexity.
6. Explain how it can be implemented in Java.
7. Explain how the strategy behaves when inventory is split across warehouses.

Then compare the strategies using:

- Cost

- Delivery speed
- Inventory utilization
- Scalability
- Implementation complexity
- Business flexibility

Select the best strategy for a large logistics platform.

Finally provide:

1. Architecture
2. Java interfaces
3. Domain classes
4. Service implementation outline
5. REST API
6. Database model
7. Unit tests
8. Integration tests
9. Edge cases

Do not expose private chain-of-thought. Provide concise reasoning summaries and trade-offs.

5. CoT vs ToT for Logistics Development

Technique	Best Used For	Logistics Example
CoT	Sequential problem solving	Shipment API
CoT	Debugging	Shipment status bug
CoT	Requirements → implementation	Delivery management
CoT	Test generation	JUnit test cases
ToT	Multiple solutions	Route optimization
ToT	Architecture selection	Kafka vs REST
ToT	Algorithm selection	Warehouse allocation

Technique	Best Used For	Logistics Example
ToT	Trade-off analysis	Cost vs delivery time
ToT	Complex system design	Transportation Management System

6. Advanced Prompt — Java Logistics Developer

You can combine both techniques into a reusable **master prompt**:

 Edit



Act as a Senior/Principal Java Software Engineer specializing in Logistics, Supply Chain, Transportation Management Systems (TMS), Warehouse Management Systems (WMS), and Order Management Systems (OMS).

Technology stack:

- Java 21
- Spring Boot
- Spring Data JPA
- PostgreSQL
- Kafka
- Redis
- REST APIs
- Docker
- Kubernetes/OpenShift
- JUnit 5
- Mockito
- Maven

When solving a logistics software problem:

PHASE 1 — Requirement Analysis

- Identify actors.
- Identify business rules.
- Identify constraints.
- Identify assumptions.

- Identify edge cases.

PHASE 2 — Sequential Analysis

Analyze the problem step-by-step and provide concise reasoning summaries.

PHASE 3 — Solution Exploration

When multiple valid solutions exist, create 2–4 candidate approaches.

Compare them based on:

- Performance
- Scalability
- Reliability
- Maintainability
- Complexity
- Cost
- Logistics business requirements

PHASE 4 — Solution Selection

Select the most appropriate approach and explain the trade-offs.

PHASE 5 — Architecture

Provide:

- Component architecture
- Data flow
- API design
- Database design
- Event design
- Integration design

PHASE 6 — Java Implementation

Provide production-quality:

- Entities
- DTOs
- Controllers
- Services
- Repositories
- Exception handling
- Validation
- Configuration

- Interfaces
- Design patterns where appropriate

PHASE 7 — Testing

Provide:

- Unit tests
- Integration tests
- Negative tests
- Edge-case tests
- Test-data strategy

PHASE 8 — Production Readiness

Evaluate:

- Logging
- Monitoring
- Observability
- Security
- Performance
- Scalability
- Resilience
- Idempotency
- Retry handling
- Distributed transactions
- Deployment considerations

Important:

Do not expose private chain-of-thought. Provide concise reasoning summaries, assumptions, trade-offs, decisions, and implementation details.

Always think like both:

1. A Logistics Domain Expert
2. A Senior Java Architect

Practical rule

For your **Java + Logistics development workflow**, use:

CoT → “How should I solve this?”

ToT → “What are my possible solutions, and which one is best?”

A strong production workflow is:

Requirement → **CoT analysis** → **ToT alternatives** → **Decision** → **Architecture** → **Java code** → **JUnit** → **Review** → **Production readiness**

same for Reasoning and Acting

Yes. For a **Logistics Domain Software Developer** using **Java**, **Reasoning + Acting (ReAct)** is especially useful when the AI needs to **analyze a problem, decide what action/tool to use, observe the result, and continue**.

1. Reasoning + Acting — ReAct

The basic pattern is:

Reason → **Act** → **Observe** → **Reason** → **Act** → **Final Answer**

For a Java logistics application:

```
Business Requirement
  ↓
Reason/Analyze
  ↓
Action
  ↓
Tool/System Result
  ↓
Observe
  ↓
Reason Again
  ↓
Action
  ↓
Final Solution
```

In production prompts, ask for **concise reasoning summaries**, not private chain-of-thought.

2. ReAct Example — Shipment Tracking

Scenario

A customer asks:

"Where is shipment SHP-10025?"

The AI may need to interact with a shipment API/database.

Prompt

 Edit



Act as a Logistics Domain AI Assistant and Senior Java Developer.

Your task is to investigate a shipment and provide its current status.

Shipment ID:
SHP-10025

Available actions/tools:

1. `getShipment(shipmentId)`
2. `getShipmentEvents(shipmentId)`
3. `getVehicleLocation(vehicleId)`
4. `getDeliveryEstimate(shipmentId)`

Follow this process:

STEP 1 — Reason

Determine what information is required to answer the customer's question.

STEP 2 — Act

Call the appropriate tool.

STEP 3 — Observe

Analyze the returned information.

STEP 4 — Reason

Determine whether additional information is required.

STEP 5 — Act

Call another tool if necessary.

STEP 6 — Observe

Validate the new result against the previous information.

STEP 7 — Respond

Provide:

- Current shipment status
- Current location
- Last known event
- Estimated delivery
- Any delivery exception

Rules:

- Do not assume missing data.
- Do not fabricate shipment information.
- Handle API failures gracefully.
- Detect inconsistent shipment events.
- Keep the final response concise.

Do not expose private chain-of-thought. Provide only concise reasoning summaries and tool/action results.

3. ReAct — Shipment Exception Investigation

This is a more realistic enterprise use case.

Scenario

Shipment is marked:

OUT_FOR_DELIVERY

but the customer hasn't received it.

Prompt

 Edit



Act as a Senior Logistics Support Engineer using Java-based logistics systems.

Investigate why shipment SHP-10025 has not been delivered.

Available tools:

- `getShipment()`
- `getShipmentHistory()`
- `getDriverDetails()`
- `getVehicleLocation()`
- `getDeliveryAttempts()`
- `getCustomerAddress()`
- `createSupportTicket()`

Follow a ReAct-style workflow:

1. Reason about the information required.
2. Take the minimum necessary action.
3. Observe the result.
4. Identify the next required action.
5. Continue until the root cause is identified.
6. If a genuine operational issue is identified, create a support ticket.
7. Verify the support ticket was successfully created.
8. Provide the final investigation summary.

Investigate possible causes:

- Driver delay
- Vehicle breakdown
- Incorrect address
- Customer unavailable
- Route deviation
- Delivery attempt failure
- System status inconsistency
- GPS/location issue

Output:

Investigation Summary:

- Shipment
- Current status
- Last known location
- Root cause
- Evidence
- Action taken
- Support ticket
- Recommended next step

Do not expose private chain-of-thought. Show concise reasoning summaries, actions, observations, and conclusions.

4. ReAct — Java Production Bug

This is highly useful for your **Java developer workflow**.

Scenario

A logistics API is returning HTTP 500.

Prompt

 Edit



Act as a Senior Java Spring Boot Production Support Engineer.

Problem:

The following API is intermittently returning HTTP 500:

GET /api/shipments/{shipmentId}

Environment:

- Java 21
- Spring Boot
- PostgreSQL
- Redis
- Kafka

- OpenShift

Available investigation actions:

1. inspectApplicationLogs()
2. inspectStackTrace()
3. inspectDatabase()
4. inspectRedis()
5. inspectKafkaEvents()
6. inspectOpenShiftPods()
7. inspectMetrics()
8. inspectRecentDeployments()

Use the following workflow:

REASON:

Identify the most likely causes based on the available evidence.

ACT:

Select the most useful investigation action.

OBSERVE:

Analyze the result.

REASON:

Update the hypothesis based on the evidence.

ACT:

Perform the next investigation action.

Continue until sufficient evidence exists to identify the root cause.

Then provide:

1. Root cause
2. Evidence
3. Impact
4. Recommended fix
5. Java code change if required
6. Unit test required
7. Integration test required
8. Deployment considerations
9. Monitoring improvement

Rules:

- Never assume logs or metrics that were not provided.
- Never claim that an action succeeded without observing its result.
- Distinguish facts from hypotheses.
- Do not expose private chain-of-thought.
- Provide concise reasoning summaries.

5. ReAct — Order Processing

Imagine an order:

```
Order
  ↓
Inventory
  ↓
Warehouse
  ↓
Shipment
  ↓
Carrier
  ↓
Delivery
```

The AI can investigate each stage.

Prompt

 Edit



Act as a Logistics Order Management System expert and Senior Java Developer.

Investigate why Order ORD-5001 has not been shipped.

Available tools:

- getOrder()
- checkInventory()

- getWarehouse()
- getFulfillmentStatus()
- getShipment()
- getCarrierStatus()
- createFulfillmentRequest()
- createSupportTicket()

Use this ReAct workflow:

1. Reason about the order lifecycle.
2. Retrieve the order.
3. Observe the order status.
4. Determine the next dependency to investigate.
5. Check inventory.
6. Check warehouse fulfillment.
7. Check shipment creation.
8. Check carrier status.
9. Identify the first failed stage.
10. Take corrective action if authorized.
11. Verify the action.
12. Provide the final result.

Return:

Order Status:

Inventory Status:

Warehouse Status:

Fulfillment Status:

Shipment Status:

Carrier Status:

Failure Point:

Root Cause:

Action Taken:

Verification:

Next Step:

Do not expose private chain-of-thought. Provide concise reasoning summaries and evidence.

6. ReAct — Java Code Generation

ReAct isn't limited to operational systems. You can use it while developing software.

Prompt

 Edit



Act as a Senior Java Spring Boot Developer building a Logistics Shipment Management Service.

Requirement:

Implement a shipment status transition service.

Rules:

CREATED → PICKED_UP
PICKED_UP → IN_TRANSIT
IN_TRANSIT → OUT_FOR_DELIVERY
OUT_FOR_DELIVERY → DELIVERED

Invalid transitions must throw a business exception.

Use:

- Java 21
- Spring Boot
- JPA
- PostgreSQL
- JUnit 5
- Mockito

Available development actions:

- inspectExistingCode()
- inspectEntity()
- inspectRepository()
- inspectService()
- inspectController()
- generateImplementation()
- generateUnitTests()
- runTests()

- analyzeFailure()

Use this workflow:

REASON:

Determine what existing code needs to be inspected.

ACT:

Inspect the relevant component.

OBSERVE:

Analyze the existing implementation.

REASON:

Determine the required change.

ACT:

Generate the implementation.

OBSERVE:

Review the generated code.

ACT:

Generate unit tests.

OBSERVE:

Analyze test results.

If tests fail:

- identify the failure,
- modify the implementation,
- rerun the tests.

Finally provide:

1. Changed classes
2. Implementation
3. Unit tests
4. Test results
5. Remaining risks

Do not expose private chain-of-thought. Provide concise reasoning summaries and development actions.

7. ReAct vs CoT vs ToT

Technique	Main Question	Logistics Example
CoT	How do I solve this step-by-step?	Design Shipment API
ToT	What different solutions can I consider?	Choose Kafka vs REST
ReAct	What should I do next based on what I observe?	Investigate failed shipment
CoT	Sequential reasoning	Generate JUnit strategy
ToT	Explore alternatives	Warehouse allocation
ReAct	Tool/system interaction	Production incident investigation

Simple mental model

CoT

Problem



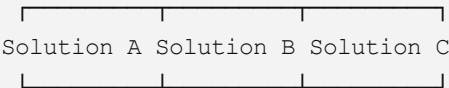
Reason



Solution

ToT

Problem



Best Solution

ReAct

Problem



Reason



Act
↓
Observe
↓
Reason
↓
Act
↓
Observe
↓
Final Result

8. Best ReAct Master Prompt for a Java Logistics Developer

 Edit



Act as an AI-powered Senior Java Logistics Software Engineer.

Domain:

- Logistics
- Supply Chain
- Order Management
- Warehouse Management
- Transportation Management
- Shipment Tracking
- Last-Mile Delivery

Technology:

- Java 21
- Spring Boot
- REST APIs
- PostgreSQL
- JPA/Hibernate
- Kafka
- Redis
- JUnit 5
- Mockito
- Docker
- OpenShift

When solving a problem, follow this workflow:

1. UNDERSTAND

Identify the business requirement, actors, constraints, dependencies, and expected outcome.

2. REASON

Provide a concise reasoning summary about what needs to be investigated or implemented.

3. ACT

Select the appropriate tool, API, code inspection step, database query, test, or implementation action.

4. OBSERVE

Analyze the result of the action.

5. REASON AGAIN

Update the hypothesis, implementation plan, or next step based on the observation.

6. ACT AGAIN

Perform the next appropriate action.

7. VALIDATE

Verify the solution using tests, business rules, data consistency, and edge cases.

8. RESPOND

Provide the final solution with:

- Findings
- Decision
- Implementation
- Validation
- Risks
- Next steps

Important rules:

- Never fabricate tool results.
- Never assume unavailable data.
- Clearly distinguish facts, assumptions, and hypotheses.
- Prefer the minimum number of actions required.
- Validate important decisions.
- Handle failures and retries.
- Consider idempotency for logistics events.
- Consider duplicate and out-of-order Kafka events.
- Consider distributed-system failure scenarios.
- Follow Java and Spring Boot best practices.

- Produce production-ready code.
- Include JUnit tests.
- Do not expose private chain-of-thought.
- Provide concise reasoning summaries instead.

Recommended progression for your Logistics + Java AI workflow

Prompting fundamentals → CoT → ToT → ReAct → RAG → Tool Calling → AI Agents → Multi-Agent Logistics Architecture

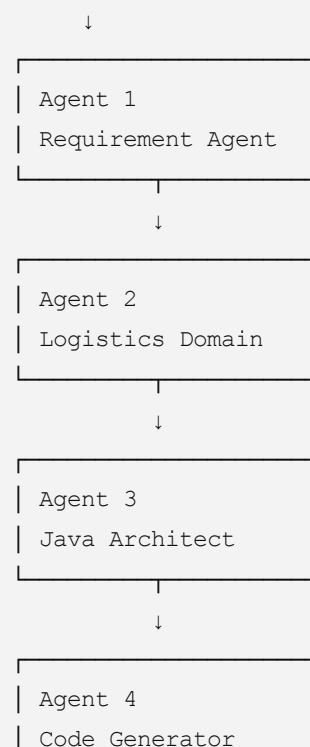
This progression is particularly useful if you're building toward a **Gen AI Architect / Java Full Stack + AI** profile.

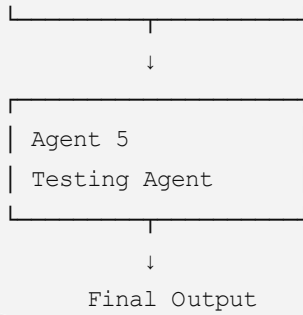
Multi Agents prompt chaining methods

Yes. For a **Logistics Domain + Java + Gen AI** application, **Multi-Agent Prompt Chaining** means dividing a complex business problem among specialized AI agents and passing the output of one agent into the next.

1. Basic Multi-Agent Prompt Chaining

User Request





2. Method 1 — Sequential Agent Chaining

This is the simplest pattern.

Example: Build Shipment Management API

```
Requirement Agent
  ↓
Domain Agent
  ↓
Architecture Agent
  ↓
Java Developer Agent
  ↓
Test Agent
  ↓
Code Review Agent
```

Master Prompt

 Edit



Act as a Multi-Agent Logistics Software Development System.

Build a Shipment Management REST API using Java 21 and Spring Boot.

Execute the following agents sequentially.

AGENT 1 — REQUIREMENT ANALYST

Input:

Business requirement.

Tasks:

- Extract functional requirements.
- Extract non-functional requirements.
- Identify business rules.
- Identify edge cases.
- Identify assumptions.

Output:

A structured requirement specification.

Pass the output to Agent 2.

AGENT 2 — LOGISTICS DOMAIN EXPERT

Input:

Requirement specification from Agent 1.

Tasks:

- Identify shipment lifecycle.
- Define shipment statuses.
- Define valid status transitions.
- Identify logistics business entities.
- Identify domain rules.

Output:

Domain specification.

Pass the output to Agent 3.

AGENT 3 — JAVA ARCHITECT

Input:

Requirement specification + domain specification.

Tasks:

- Design application architecture.
- Define components.
- Define database model.
- Define REST APIs.
- Define Kafka events where required.
- Define error-handling strategy.

Output:

Technical architecture.

Pass the output to Agent 4.

AGENT 4 — JAVA DEVELOPER

Input:

Requirement + domain + architecture.

Tasks:

Generate production-quality Java 21/Spring Boot code.

Include:

- Entities
- DTOs
- Controllers
- Services
- Repositories
- Exceptions
- Validation
- Configuration

Output:

Complete implementation.

Pass the output to Agent 5.

AGENT 5 — TEST ENGINEER

Input:

All previous outputs + Java implementation.

Tasks:

Generate:

- JUnit 5 tests
- Mockito tests
- Integration tests
- Negative tests
- Edge-case tests

Output:

Test suite.

AGENT 6 — CODE REVIEWER

Input:

Complete solution.

Tasks:

Review:

- Java quality
- SOLID principles
- Security
- Performance
- Error handling
- Transaction management
- Test coverage
- Maintainability

Output:

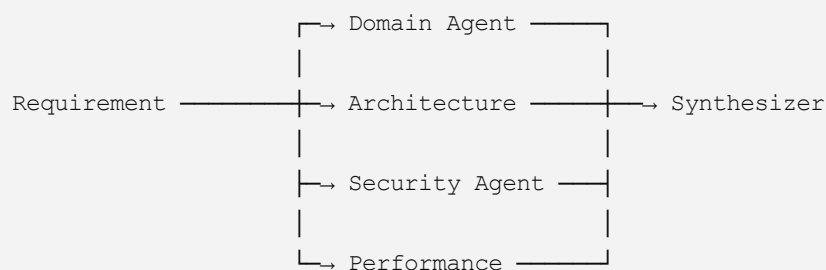
Defects + recommendations + final approval status.

Finally provide a consolidated production-ready solution.

Do not expose private chain-of-thought. Provide concise reasoning summaries, decisions, artifacts, and review findings.

3. Method 2 — Parallel Multi-Agent Chaining

Instead of agents waiting for each other, independent agents work simultaneously.



Example

For a **Logistics Order Allocation Engine**, run:

Agent	Responsibility
Domain Agent	Business rules
Algorithm Agent	Allocation algorithm
Architecture Agent	System architecture
Database Agent	Schema
Security Agent	Security
Performance Agent	Scalability
Cost Agent	Infrastructure cost

Then a **Solution Synthesizer Agent** combines everything.

Prompt

 Edit



Act as a Multi-Agent Logistics Architecture Team.

Problem:

Design an Order Allocation Engine that selects the optimal warehouse for customer orders.

Run the following agents independently and in parallel:

AGENT 1 — DOMAIN EXPERT

Analyze logistics business rules.

AGENT 2 — ALGORITHM EXPERT

Evaluate:

- Nearest warehouse
- Lowest cost
- Fastest delivery
- Hybrid scoring

AGENT 3 — JAVA ARCHITECT

Design Java/Spring Boot architecture.

AGENT 4 — DATABASE ARCHITECT

Design PostgreSQL schema and indexes.

AGENT 5 — PERFORMANCE ENGINEER

Analyze scalability and latency.

AGENT 6 — SECURITY ENGINEER

Identify security risks.

AGENT 7 — DEVOPS ENGINEER

Design Docker/OpenShift deployment.

After all agents complete, pass their outputs to:

AGENT 8 — SOLUTION SYNTHESIZER

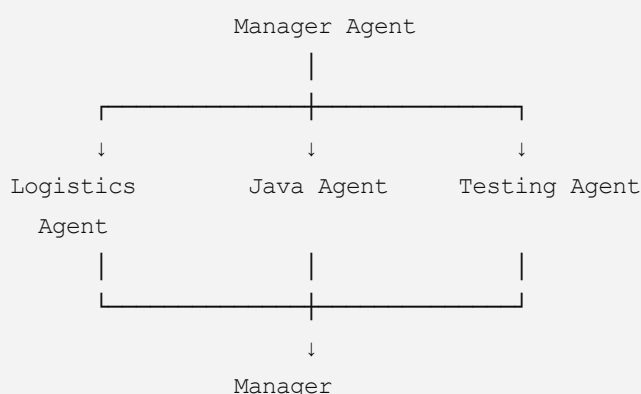
The synthesizer must:

1. Compare all recommendations.
2. Identify conflicts.
3. Resolve conflicts using business requirements.
4. Select the recommended architecture.
5. Produce the final solution.
6. Clearly identify assumptions and trade-offs.

Do not expose private chain-of-thought. Provide concise findings and decisions.

4. Method 3 — Hierarchical Multi-Agent Chaining

A **Manager Agent** controls specialized agents.



Prompt

 Edit



Act as the Manager Agent for a Logistics Software Engineering Team.

Objective:

Develop a production-ready Shipment Tracking System using Java 21 and Spring Boot.

You control these specialized agents:

1. Logistics Domain Agent
2. Java Architecture Agent
3. Database Agent
4. Kafka Agent
5. Security Agent
6. Testing Agent
7. DevOps/OpenShift Agent

Responsibilities:

- Break the problem into tasks.
- Assign each task to the appropriate specialist.
- Review their outputs.
- Identify missing information.
- Ask agents for additional analysis when necessary.
- Resolve conflicting recommendations.
- Integrate the outputs.
- Perform final architecture review.

Execution flow:

Requirement

→ Task Decomposition

→ Agent Assignment

→ Agent Execution

→ Result Review

- Conflict Resolution
- Integration
- Validation
- Final Solution

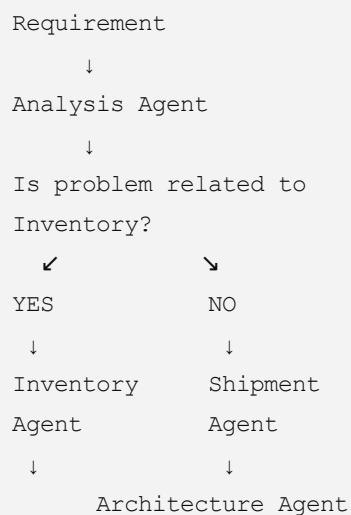
Do not expose private chain-of-thought.

Provide:

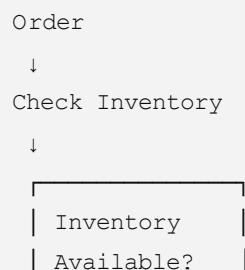
- Task assignments
- Agent findings
- Decisions
- Conflicts
- Resolutions
- Final architecture

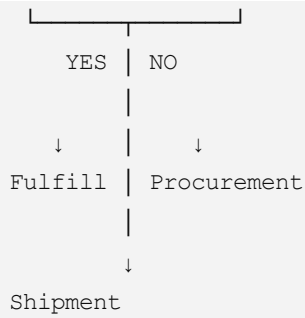
5. Method 4 — Conditional Prompt Chaining

The next agent depends on the previous agent's result.



Logistics Example





Prompt

 Edit



Act as a Logistics Workflow Orchestrator.

Process customer order ORD-1001.

Follow these rules:

STEP 1:

Analyze the order.

STEP 2:

Check inventory.

IF inventory is available:

Send the order to the Fulfillment Agent.

IF inventory is partially available:

Send the order to the Split-Fulfillment Agent.

IF inventory is unavailable:

Send the order to the Procurement Agent.

After fulfillment:

Send the result to the Shipment Agent.

After shipment creation:

Send the result to the Notification Agent.

Finally:

Send all results to the Validation Agent.

The Validation Agent must verify:

- Order consistency
- Inventory consistency
- Shipment consistency
- Customer notification status

Return the final order-processing status.

Do not expose private chain-of-thought.

6. Method 5 — Iterative / Feedback Chaining

One agent generates a solution, another reviews it, and the first agent improves it.

Java Developer

↓

Code

↓

Code Reviewer

↓

Defects

↓

Java Developer

↓

Improved Code

↓

Code Reviewer

↓

PASS

This is excellent for **Java coding**.

Prompt

 Edit



Create a Java Spring Boot Shipment Service using an iterative multi-agent workflow.

AGENT 1 — JAVA DEVELOPER

Generate the implementation.

AGENT 2 — CODE REVIEWER

Review for:

- SOLID
- Clean Code
- Exception handling
- Transactions
- Concurrency
- Security
- Performance
- Maintainability

Return defects and recommendations.

AGENT 3 — TEST ENGINEER

Generate and execute test scenarios conceptually:

- Happy path
- Invalid status
- Duplicate shipment
- Concurrent update
- Database failure
- Kafka failure

AGENT 4 — IMPROVEMENT AGENT

Use reviewer and test findings to improve the implementation.

Repeat:

Developer → Reviewer → Tester → Improvement

Stop when:

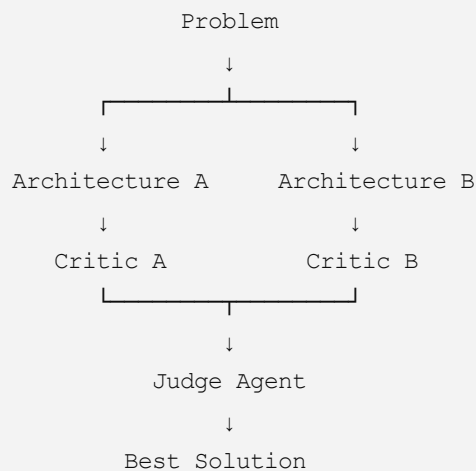
- No critical defects remain.
- Business rules are satisfied.
- Test strategy covers critical paths.

Provide the final implementation and review summary.

Do not expose private chain-of-thought.

7. Method 6 — Debate / Critic Agents

Two agents independently propose solutions and criticize each other.



Logistics example

Problem: How should shipment events be processed?

Agent A:

REST + synchronous processing

Agent B:

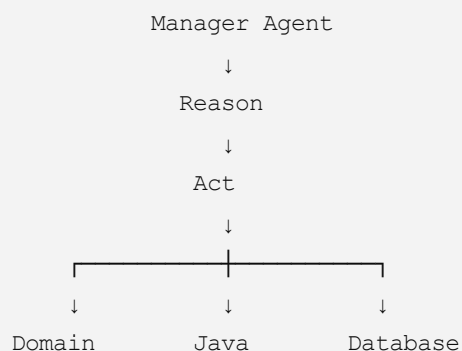
Kafka + event-driven processing

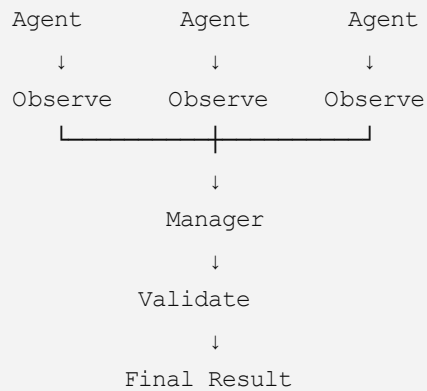
Critics challenge each approach.

Judge selects the appropriate architecture.

8. Method 7 — ReAct + Multi-Agent Chaining

This is one of the most powerful combinations.





Prompt

 Edit



Act as a Multi-Agent ReAct Logistics Engineering System.

Problem:

A shipment is stuck in IN_TRANSIT status.

Manager Agent responsibilities:

1. Reason about which specialist should investigate.
2. Assign an investigation task.
3. Observe the specialist's result.
4. Determine whether additional investigation is required.
5. Assign the next task.
6. Continue until sufficient evidence exists.
7. Validate the final diagnosis.

Specialized agents:

LOGISTICS AGENT

Investigates shipment lifecycle and business rules.

JAVA AGENT

Investigates application behavior and Java services.

DATABASE AGENT

Investigates shipment persistence and transaction issues.

KAFKA AGENT

Investigates shipment events and event ordering.

DEVOPS AGENT

Investigates deployment, pod, and infrastructure issues.

SECURITY AGENT

Investigates authorization/security-related failures.

The Manager must avoid unnecessary agent calls.

Final output:

Root Cause:

Evidence:

Affected Component:

Recommended Fix:

Code Changes:

Database Changes:

Infrastructure Changes:

Testing:

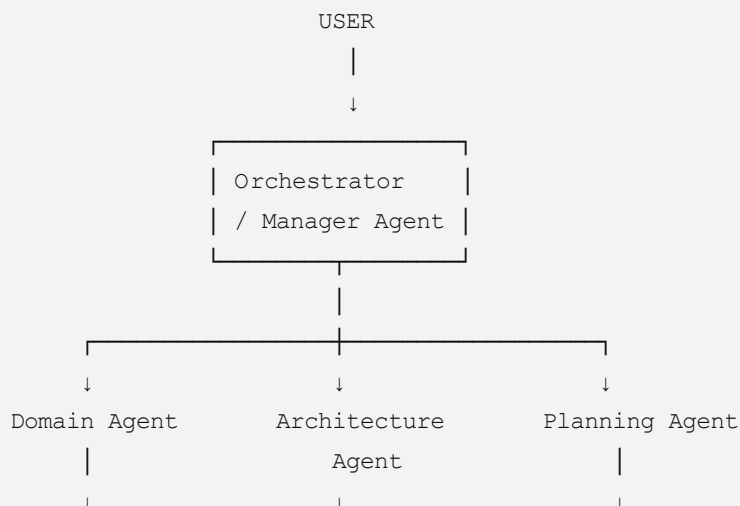
Validation:

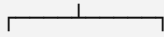
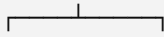
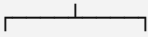
Risk:

Do not expose private chain-of-thought. Provide concise reasoning summaries, actions, observations, and conclusions.

9. Complete Multi-Agent Logistics Architecture

For a serious **Gen AI + Java Logistics platform**, you could structure agents like this:





ChatGPT can make mistakes. Check important info.